At this point, your original integration test is still failing because you haven't integrated the controller with the new model function. The controller method still calls Project.all, but you can fix that:

security/05/app/controllers/projects_controller.rb

```
def index
  @projects = current_user.visible_projects
end
```

Now the new integration test passes, but you have an interesting regression failure:

```
adding a project allows a user to create a project with tasks
     Failure/Error:
       expect(page).to have_selector(
           "#project_#{@project.id} .name", text: "Project Runway")

       expected to find visible css "#project_1 .name" with text
       "Project Runway" but there were no matches
     # ./spec/system/add_project_spec.rb:18:in `block (2 levels) in
     <top (required)>'
```

The test in question is an integration test that creates a new project using CreatesProject and then goes to view the page. The test is failing because the user isn't added as a member of the new project and therefore can't see the project page.

Fixing this involves changing the CreatesProject action to take a user (or users) and add that user to the project when the project is created. Since CreatesProject is a workflow, you can isolate the test and just make sure that a passed user is applied to the project—or in this case, set the API to take an array of users:

security/05/spec/workflows/creates_project_spec.rb

```
it "adds users to the project" do
  user = create(:user)
  creator = CreatesProject.new(name: "Project Runway", users: [user])
  creator.build
  expect(creator.project.users).to eq([user])
end
```

Then add the new keyword argument to the action and use the value when creating the new project:

security/05/app/workflows/creates_project.rb

```
class CreatesProject
  attr_accessor :name, :project, :task_string, :users

  def initialize(name: "", task_string: "", users: [])
    @name = name
    @task_string = task_string || ""
```